

Software Implementation and Testing Document

For

Group 10

Version 3.0

Authors:

Kirra Orndorff
Alexander Jubran
Lauren Chang
Riley Galpin
Michael Greenberg

1. Programming Languages (5 points)

C#: This is our primary language for application logic. We chose C# because it is the native language for the .NET MAUI framework, allowing us to share a single codebase across multiple platforms while maintaining high performance.

XAML: Used in conjunction with C# within the .NET MAUI framework to define the user interface. It allows for a clean separation between the UI design and the underlying business logic (MVVM pattern).

2. Platforms, APIs, Databases, and other technologies used (5 points)

Platform: .NET MAUI: We are using this framework to build a cross-platform application that can run as a web application while retaining the ability to deploy to mobile devices in the future. We will use .NET SDK version 8.0.

Weather API ([weatherAPI](#)): This API is used to retrieve real-time environmental data including temperature, wind speed, and humidity to power our clothing recommendation engine.

.NET Geolocation: To be used in conjunction with the WeatherAPI. It provides APIs to retrieve the device's current geolocation coordinates.

Google Maps API: Integrated to allow users to view, create, and save custom running routes with specific terrain data.

OpenRouteService: for routing API

OpenStreetMapAPI: contributors for map data

MapLibreAPI: for the mapping library

Turf.jsAPI: for geospatial calculations

Google CalendarAPI: for weekly planner and past workout log.

Local Storage (.NET MAUI Built-in): We are utilizing the framework's internal storage capabilities to ensure user privacy and provide offline access to saved routes and shoe mileage.

UraniumUI.Material & Community Toolkit.Mvvm: These libraries are used to enhance the UI experience and implement the Model-View-ViewModel architecture for better code maintainability.

3. Execution-based Functional Testing (10 points)

The NWS integration was tested by connecting the API to our application and confirming that the data on our application matched that of the Tallahassee weather listed on the NWS website. We also manually changed the zip code of the weather finder within our app to test different locations and confirmed that the correct data was being sent. This was also used to test the clothing recommendation engine. Based on what weather was being sent in, we confirmed that the clothing that was recommended to the user would accurately reflect that of their cold sensitivity and the weather. The route recommendation was tested again by changing the weather to make sure that the routes that were recommended to the user were correctly based on the algorithm. For example, if it was raining, we made sure that the recommended routes were on asphalt or indoors. The route management feature was tested by adding, editing, and deleting routes, making sure that these changes persisted across the application. The shoe closet and mileage

tracker were both tested in ways similar to that of route management. The Weekly planner view was tested by adding and deleting workouts and making sure that these changes were reflected accurately in the application, as was done with the performance history in the profile section.

4. Execution-based Non-Functional Testing (10 points)

The data security and algorithm performance were tested by ensuring that the data was accurate and persistent while also being secure. The algorithm performance was confirmed to load in under 2 seconds via testing.

5. Non-Execution-based Testing (10 points)

This increment was mostly about connecting all of the pages together and making sure that data was accurately saved and displayed across pages. This was tested by going through complete user stories, such as adding a route, then using that route to add a workout, then adding a PR from that workout. Edge cases like adding negative mileage were also tested to ensure that there are error bounds in place for the application.